

ArabicaAI — V1 Architecture, Capacity and Budget

Proposal for Lucas • September 28, 2026 • USD, excluding taxes

Decision in one sentence

Build a web application with separate frontend, API and database components, reproducible locally; publish the demo on a single VM if it helps the presentation. Demonstrate capacity through testing and state the limits. Actual production traffic remains unknown.

1. What the organizers' responses change

The shared messages allow local execution and ask for a credible path to production. They do not require processing an unknown share of a real bank's traffic or spending money on cloud infrastructure. Our commitment is a functional banking journey with access controls, persistence, measurement and failure handling.

Customer promise: "I can report the correct charge, receive an accepted case reference, and understand the next steps." This excludes dispute resolution, refunds, card blocking and fraud determination.

2. Recommended stack

Reuse the team's familiarity with Angular, FastAPI and PostgreSQL to reduce delivery risk.

Component	Choice	Rationale
-----------	--------	-----------

Frontend	Angular + TypeScript; static build	Reuse existing familiarity. React + Vite is an alternative if the team already has a working starter; do not build two interfaces.
Backend	FastAPI + Pydantic + SQLAlchemy + Alembic	Explicit contracts, validation and migrations
Persistence and serving	PostgreSQL	Cases, sessions, idempotency, events and an indexed subset of transactions
Data preparation	Manoella's DuckDB + Parquet pipeline	Run outside request handling; publish a validated data version
Conversation	Direct provider SDK + structured output	Extract intent and fields; keep decisions and permissions in the backend
Runtime	Docker Compose; Caddy/Nginx proxy	Same service layout locally and on the VM
Verification	pytest integration tests, Playwright for the journey, k6 for load	Correctness, visible behavior and measured capacity

Update to the earlier recommendation: SQLite remains viable for a single-instance demo. For a web application with multiple sessions, I recommend PostgreSQL from the start. This simplifies evolution; it is not evidence that the workload requires a distributed database.

Frontend → proxy → FastAPI → PostgreSQL.

FastAPI → LLM provider, with timeouts and call limits.

DuckDB/Parquet → versioned, validated load → serving tables.

Do not add Kubernetes, Kafka, Redis or a vector database without a demonstrated need. Do not run the entire medallion pipeline on the small demo VM during the presentation.

Three screens are enough

- **Customer service:** conversation and structured transaction candidates, including currency, status and time reference.
- **Review:** customer checks the transaction and report, then approves; show a reference only after confirmed acceptance.
- **Agent:** case list and details with evidence, verified actions and unresolved questions. This is the sandbox receiving service, not a real bank integration.

Minimum contracts

- **search_transactions:** backend-validated session identity; typed filters; candidates, data version/coverage and explicit search state.
- **create_case:** revalidate ownership, bind approval to the confirmed content, use a unique idempotency key and a database transaction.
- **get_case:** authorized retrieval of the persisted case reference and status.
- Backend-issued/validated test sessions and a separate agent role. Entering a customer_id does not authenticate anyone.
- No false success references on failure; bounded retries; access only to necessary columns. Keep fraud labels outside the decision path.

- Store sessions and conversation state in the database, not only in process memory.

3. What to investigate before sizing

The profile reports 686,296 interactions and 1,097 distinct processing dates: dividing them yields approximately **625.6 interactions/day**. This is not yet a validated event-date average and does not establish the 780–900/day cited in the discussion. Check the time range, channels and excluded days.

The call_center_interactions table includes calls, chat, email and video. An interaction row is neither an HTTP request nor a concurrent session.

Investigation deliverable

Measure	How to obtain it	Decision supported
Coverage	Min/max interaction_date, missing/invalid dates and days without records; compare process_date	Avoid averaging over the wrong period
Demand	Daily/hourly counts by interaction_type and channel; report 2024 and 2025 separately	Identify peaks and workload composition

Target intake	Filter complaints.subcategory = 'Cargo no reconocido', group by creation_date and reception_channel	Describe target demand; do not extrapolate from total traffic without evidence
Serving	Rows/bytes of the validated subset, indexes, candidates per customer and search latency	Memory, disk and query needs
New application	Requests, turns, LLM calls/tokens, duration and writes per episode	Convert episodes into workload and cost

Do not infer production population from a sample with an unknown sampling fraction. Do not join complaints to interactions through origin_interaction_id: it is empty. Confirm timezone before interpreting peak hours; recorded time is not necessarily local time in each country.

Starter SQL — DuckDB; adapt names to the Silver contract

These examples have not been executed against the repository. Use Bronze for reconciliation and validated Silver for the final estimate.

```

-- 1. Check coverage and parsing failures.
SELECT
  count(*) AS rows,
  count(DISTINCT interaction_id) AS unique_interactions,
  min(TRY_CAST(interaction_date AS TIMESTAMP)) AS first_event,
  max(TRY_CAST(interaction_date AS TIMESTAMP)) AS last_event,
  count(*) FILTER (
    WHERE TRY_CAST(interaction_date AS TIMESTAMP) IS NULL
  ) AS invalid_or_missing_event_date
FROM bronze.call_center_interactions;

-- 2. Daily series for 2024–2025, explicitly including zero-record days.
WITH calendar AS (
  SELECT CAST(d AS DATE) AS day
  FROM generate_series(
    DATE '2024-01-01', DATE '2025-12-31', INTERVAL 1 DAY
  ) AS t(d)
), daily AS (
  SELECT CAST(TRY_CAST(interaction_date AS TIMESTAMP) AS DATE) AS day,
    count(*) AS n
  FROM bronze.call_center_interactions
  GROUP BY 1
), series AS (
  SELECT c.day, coalesce(d.n, 0) AS n
  FROM calendar c LEFT JOIN daily d USING(day)
)
SELECT year(day) AS year,
  sum(n) AS interactions, avg(n) AS daily_mean,
  quantile_cont(n, 0.95) AS
  daily_p95,
  max(n) AS daily_max,
  count(*) FILTER (WHERE n = 0) AS zero_days
FROM series GROUP BY 1 ORDER BY 1;

```

Zero may indicate missing coverage rather than no demand: reconcile against the ingestion manifest. Then repeat by channel/type and build an hourly series including empty hours.

4. Capacity: test assumptions, not forecasts

Start with 100, 900 and 9,000 episodes/day as exploratory scenarios. None represents confirmed production traffic or proven VM capacity.

Editable example: 30 days/month, 10 requests/episode, 10% of episodes arriving during the peak hour and 2 seconds/request.

- Monthly requests = episodes/day × 30 × requests/episode.
- Approximate peak RPS = episodes/day × peak-hour fraction × requests/episode ÷ 3,600.
- Approximate concurrent requests = RPS × average service time in seconds.
- Open sessions are a different measure; reading/thinking time is not CPU usage.
- Long LLM calls increase in-flight requests. Measure retrieval, database and model separately.

These are average approximations, not sizing guarantees for bursts or tail latency. Run k6 with arrival patterns, ambiguity, failures and retries. First use a simulated LLM to isolate the backend; then run a small test with the actual model to observe cost, latency and quotas.

Report p50/p95 latency, throughput, error rate, duplicates, isolation, CPU/RAM, database connections and token usage. Claimed capacity should be the highest measured workload meeting the agreed criteria, with test duration and configuration stated. Do not infer capacity from vCPU count or daily averages.

5. AWS budget: single-instance demo

Public pricing reference consulted for the original document on September 28, 2026. Planning region: US East (N. Virginia), subject to confirmation when provisioning and to event rules. Region, latency and data residency require a separate production decision.

Item	Assumption	USD/month
------	------------	-----------

Lightsail Linux with IPv4	2 vCPU, 4 GB RAM, 80 GB SSD; published bundle includes 4 TB transfer	24.00
Snapshots	20 billable GB-month × 0.05; an assumption, not a guaranteed backup size	1.00
Optional object storage	5 GB/25 GB transfer bundle; a small backup copy	1.00
Subtotal above	Before credits, overages and LLM	26.00

PostgreSQL runs on the same VM for the demo; no separate managed database charge is included. The 4 GB configuration is a starting point to test, not a guarantee. The 2 GB bundle is listed at USD 12/month; reduce only after measuring memory. Database, indexes and backups must fit on disk.

Excluded: LLM, domain, taxes, transfer overages, extra storage, paid monitoring, operational labor, high availability and cloud batch processing. USD 26 is a subtotal, not the total product cost. Hosting the application and database together creates a single point of failure.

Free tier and credits

The AWS page consulted for the original document advertised USD 100 initially and up to USD 100 additional credits for new customers, for up to six months. Under the Free plan, the account closes when the period or credits run out; charging requires upgrading to Paid. Confirm eligibility and available services in the account before relying on this. Do not automatically apply the

older “12 months free” rules.

Show gross cost and credits separately. Temporary credit is not permanent savings. Budget alerts notify; they are not, by themselves, hard spending caps. Also limit LLM calls, output size and demo exposure; plan shutdown and resource removal after the event.

Official calculator and attached spreadsheet

- AWS Pricing Calculator: <https://calculator.aws/>
- For this bundle, the Lightsail pricing table is the direct reference. If the service is absent from the calculator catalog, record it as a manual item; do not silently substitute EC2.
- Record region, resources, hours, storage, traffic and retention; keep credit separate from gross price. Export/share the official estimate after filling in the available services.
- **No saved estimate was created in the official calculator.** The attached CSV is an editable calculation worksheet with formulas and public prices for the verified items.
- For the LLM, enter the selected model's price per million tokens. Count ALL calls, resent conversation history, retries and evaluations. Without a selected model and its prices, the total remains incomplete.

6. AWS or GCP?

Recommendation for this week: local first → optional Lightsail, for a predictable demo infrastructure subtotal and Compose portability. If the team already has working GCP access/credits, familiarity may matter more.

A managed alternative is a static frontend + Cloud Run/FastAPI + Cloud SQL/PostgreSQL. Cloud Run can reduce idle compute, but the database,

networking, builds, images and logs have separate costs. Do not store SQLite/cases on Cloud Run's ephemeral filesystem. Set instance and connection-pool limits before enabling scaling.

GCP prices/free tier could not be reliably retrieved during the original research, so this document includes neither a GCP total nor a zero-cost promise. Populate <https://cloud.google.com/products/calculator> with region, CPU/memory, request duration/count, minimum instances, database/HA, storage, backups, logs and egress before comparing.

The production path separates a managed database, stateless application, real identity service, secrets management, restore-tested backups, monitoring and redundancy. Adopt HA/autoscaling according to SLOs, RTO/RPO, traffic and testing—not merely because the hackathon mentions scalability.

7. How to start

First block: confirm contracts with Manoella; start frontend, API and PostgreSQL in Compose. Publish a validated data subset. Implement search, review, idempotent acceptance and the agent screen without an LLM.

Second block: connect the model to the same tools used by the checklist. Record all attempts, including failures and abandonment, with versions and tokens.

Third block: test cross-customer access, failed writes, repeated confirmation and ES/PT cases; measure load and complete the worksheet.

Only then: publish the demo if beneficial, retaining local execution as a fallback.

The first milestone is a test customer registering a case that the agent can still see after an API restart. The product evidence is the complete journey; scalability evidence is measurement and an explanation of the next limiting factor.

Sources and limitations

- Bronze profile and Roberto's review shared in the conversation; no execution against the repository.
- Organizer messages shared by Lucas; scope interpretation described above.
- AWS Lightsail: <https://aws.amazon.com/lightsail/pricing/> — consulted for the original document on September 28, 2026.
- AWS Free Tier: <https://aws.amazon.com/free/> — consulted for the original document on September 28, 2026.
- AWS Calculator: <https://calculator.aws/>
- GCP Calculator: <https://cloud.google.com/products/calculator> — provided for estimation; no validated GCP quote in this document.

English edition of the original proposal. Pricing assumptions are preserved, not newly re-quoted.